Modeling Human Activity States Using Hidden Markov Models

Dan Paul Dushime · Machine Learning Techniques II · African Leadership University · 29 October 2026 (individual work)

GitHub repository (dataset, notebook, videos): https://github.com/<your-username>/<repo-name>

1. Background and Motivation

Students at ALU spend long days at a desk, and the only device they carry all day is a smartphone. My use case is **a phone-only movement tracker for students** that estimates active minutes and detects long inactive periods without buying a wearable. To do this it must tell apart four states: the phone set aside on a surface (**Still**), the student upright but idle (**Standing**), and active movement (**Walking**, **Jumping** as a short burst of exercise). The true activity is never observed directly; the phone only delivers noisy accelerometer and gyroscope readings, and activities persist over time and change in plausible orders (a phone is picked up before anyone walks with it). A Hidden Markov Model fits this problem exactly: hidden states are the activities, observations are features of short sensor windows, and the transition matrix encodes how behaviour evolves, which lets the model correct isolated noisy windows using temporal context.

2. Data Collection and Pre-processing

2.1 Recording protocol

All data were collected by me with **Sensor Logger v1.68 (Android) on a Google Pixel 3a**, recording accelerometer (linear acceleration, gravity removed), gyroscope and total acceleration with nanosecond timestamps, on 28–29 October 2026 in Kigali. For Standing, Walking and Jumping the phone was held flat at waist level; for Still it was placed flat on the floor. Each activity was recorded **50 times for 5–10 s**, giving 200 labelled files and more than the required 1 min 30 s per activity.

Table 1 – Dataset after cleaning (durations measured after trimming)

Activity	Files	Total duration	Mean clip	Video evidence
Still (phone on floor)	50	346 s (5 min 46 s)	6.9 s	<repo>/blob/main/videos/still.mp4
Standing	50	294 s (4 min 54 s)	5.9 s	<repo>/blob/main/videos/standing.mp4
Walking	50	398 s (6 min 38 s)	8.0 s	<repo>/blob/main/videos/walking.mp4
Jumping	50	358 s (5 min 58 s)	7.2 s	<repo>/blob/main/videos/jumping.mp4

2.2 Sampling rates and harmonisation

Even on a single phone the sensors run at different rates, so they cannot be combined sample-by-sample:

Table 2 – Sampling rate per member

Member	Device	Accelerometer (native)	Gyroscope (native)	After harmonisation
Dan Paul Dushime	Google Pixel 3a (Android 12L)	50.4 Hz	100.8 Hz	50 Hz (both)

Every recording is **resampled onto a common 50 Hz time grid by linear interpolation**, restricted to the interval where all sensors overlap. 50 Hz is the lowest native rate, so the accelerometer is never up-sampled (no invented data), and its Nyquist frequency of 25 Hz is well above human movement frequencies (< 20 Hz), so down-sampling the gyroscope loses no useful information. The same code works for any phone: whatever a device's native rates, its output is always 50 Hz, so data from different phones would be directly comparable.

2.3 Cleaning

Samples are sorted by timestamp and duplicate timestamps dropped; **0.5 s is trimmed at both ends** of each clip to remove the tap that starts/stops the recording; clips are capped at 10 s. No timestamp gaps were found (maximum gap 20 ms at 50 Hz). Each cleaned recording is saved as dataset/<activity>/<activity>_NN.csv with columns timestamp_ns, seconds_elapsed, acc_x/y/z, gyro_x/y/z and tacc_x/y/z. Figure 1 shows one cleaned clip per activity.

2.4 Windowing

Features are computed on windows of **128 samples = 2.56 s with 50 % overlap (hop 64 samples = 1.28 s)**. The size follows from the 50 Hz rate: 2.56 s contains at least 2–4 complete steps or jumps (cadence ≈ 1.5–2.5 Hz), so each window captures the periodic pattern, and the FFT resolution is 50/128 ≈ 0.39 Hz, fine enough to separate walking and jumping rhythms. A power-of-two length keeps the FFT efficient, and even the shortest clip still yields 2 windows (most give 3–6). This produced **794 windows**.

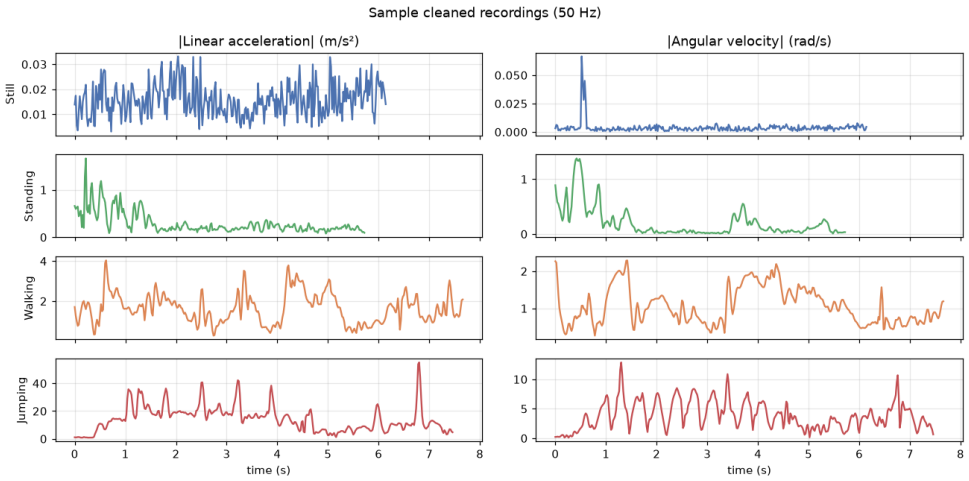


Figure 1 – Cleaned 50 Hz signals: magnitude of linear acceleration (left) and angular velocity (right).

2.5 Feature extraction and normalisation

Table 3 – Extracted features (9 per window: 6 time-domain, 3 frequency-domain from the FFT)

Feature	Domain	Why it separates the activities
acc_std – std of linear-acc magnitude	Time	≈ 0 when still, small when standing, large walking, very large jumping
acc_sma – signal magnitude area of acc	Time	overall movement intensity, robust to the phone's orientation
gyro_std, gyro_sma – rotation std / SMA	Time	body sway and rotation: zero on the floor, present even when standing
acc_corr_yz – correlation of acc y and z	Time	coordinated vertical/forward motion in jumps and gait vs random noise
tilt_z – gravity on z / |g| (mean)	Time	phone orientation: ≈ 1 when flat (Still, Standing), drops when it swings (Jumping)
dom_freq – dominant FFT frequency	Freq.	step / jump cadence; random for static states
spec_energy – FFT spectral energy	Freq.	energy of the motion: spans 6 orders of magnitude from Still to Jumping
spec_entropy – normalised spectral entropy	Freq.	periodic motion (low entropy) vs sensor noise (high entropy)

Two transformations make the features suitable for Gaussian emissions. (1) **Drift removal:** Android's linear acceleration subtracts an estimated gravity vector; after the phone is moved quickly that estimate lags for several seconds and leaves a slowly decaying offset although the phone is motionless. Each accelerometer axis is therefore linearly detrended inside every window. (2) Intensity features (std, SMA, spectral energy) are **log10-transformed**, because their raw distributions are extremely skewed. Finally all features are **Z-score normalised**, (x − μ)/σ, with μ and σ computed on the training set only and re-used for validation and test data. Z-scoring puts all features on one scale so that no feature dominates the covariance matrices, it matches the Gaussian emission assumption, and unlike min–max scaling it is not driven by a single extreme jumping window. Figure 2 confirms that the intensity features separate the four activities almost perfectly, while dom_freq and acc_corr_yz add complementary information.

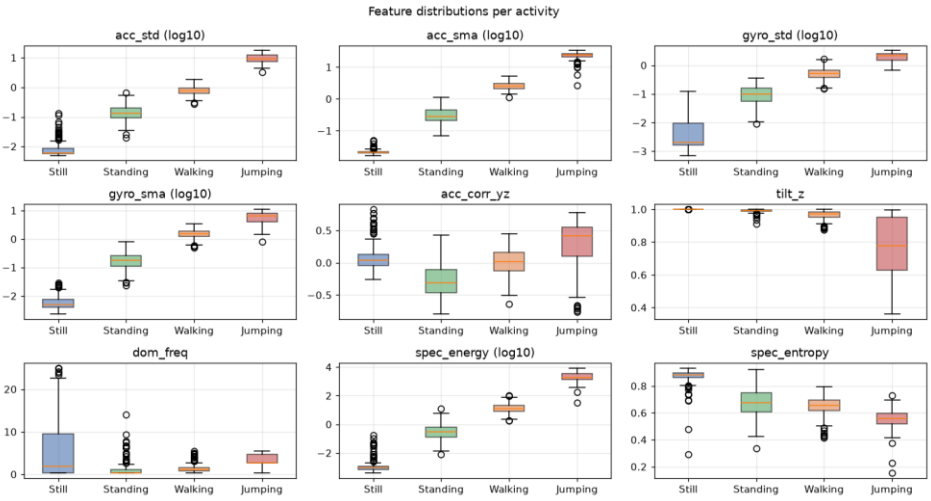


Figure 2 – Distribution of each feature per activity (log10 where noted).

3. HMM Setup and Implementation

Table 4 – HMM components

Element	Definition in this project
Hidden states Z	Still, Standing, Walking, Jumping (K = 4)
Observations X	9-dimensional Z-scored feature vector of each 2.56 s window
Transitions A	A[i, j] = P(z_t = j | z_t−1 = i) between consecutive windows (1.28 s apart)
Emissions B	multivariate Gaussian N(x | μ_k, Σ_k) per state, full covariance (+0.01·I for stability)
Initial probabilities π	π_k = P(z_1 = k)

Training data. Recordings were split **by file** into 80 % training (160 clips, 637 windows) and 20 % validation (40 clips, 157 windows), so overlapping windows of one clip never appear on both sides. Because each clip contains one activity, training clips were chained into 28 sessions that start with the phone Still or Standing and follow physically plausible transitions (e.g. Still → Standing → Walking → Jumping → Standing → Still); a phone cannot go from lying on the floor to jumping without first being picked up. This gives Baum–Welch real activity changes to learn A from.

Algorithms (tied to the evaluation metrics). Baum–Welch (EM with the forward–backward algorithm) learns π, A, μ and Σ by maximising the likelihood of the training sequences. It is initialised from the labelled windows (class means/covariances and smoothed transition counts), which anchors each hidden state to one activity so that predictions can be scored. Training stops when the total log-likelihood improves by less than $\varepsilon = 10^{-3}$ (maximum 200 iterations); it **converged after 7 iterations** (614.97 → 736.80, Figure 3). **Viterbi** decoding returns the single most probable state sequence, i.e. one label per window, which is exactly what sensitivity, specificity and accuracy are computed from. Because Viterbi uses A, a single noisy window cannot flip the label unless its evidence outweighs the cost of two unlikely transitions, which reduces isolated errors. The **forward algorithm** provides the log-likelihood for the convergence check.

Implementation. The model is written from scratch in NumPy as a GaussianHMM class with methods init_supervised, log_emission (Cholesky-based Gaussian log-density), forward, backward, fit (Baum–Welch), viterbi and predict_independent (an emission-only baseline). All recursions run in log space with log-sum-exp to avoid underflow, and A and π are floored at 10^{-4} so that no transition becomes impossible on unseen data. The full pipeline runs in seconds.

4. Results and Interpretation

4.1 Learned parameters

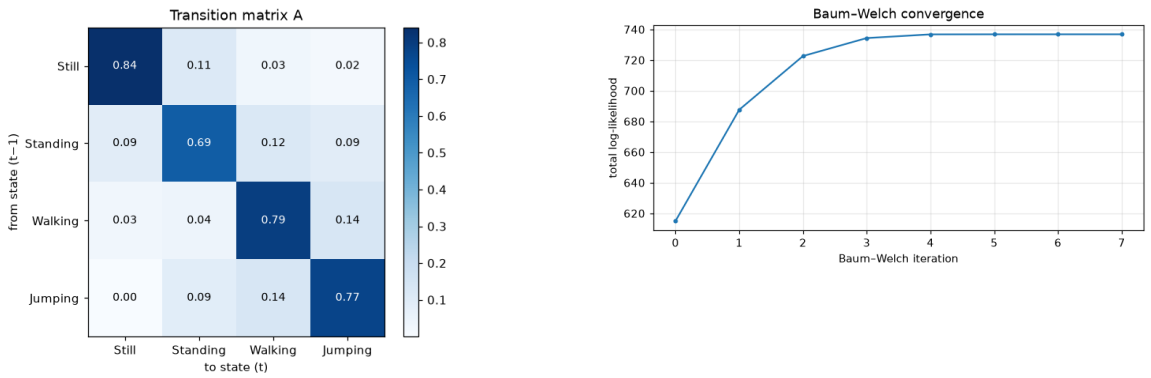


Figure 4 – Learned transition matrix A (left) and Baum–Welch log-likelihood per iteration (right).

The transition matrix is strongly diagonal (0.69–0.84): activities persist. With a hop of 1.28 s the expected stay 1.28/(1 − A_ii) is 8.0 s for Still, 4.2 s Standing, 6.2 s Walking and 5.7 s Jumping, consistent with the 5–10 s clips. The learned initial distribution is π = (Still 0.82, Standing 0.18, Walking 0.00, Jumping 0.00), reflecting sessions that begin with the phone at rest. The emission densities (Figure 5) show well-separated Gaussians on acc_std and gyro_std, with Standing between Still and Walking, and tilt_z isolating Jumping (the full emission-mean heatmap is in the notebook).

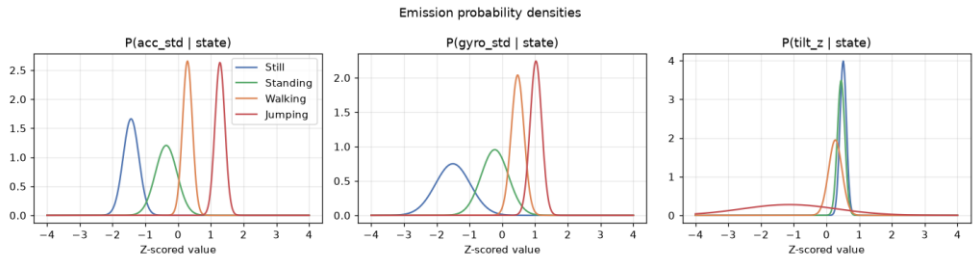


Figure 5 – Emission probability densities P(feature | state) for three key Z-scored features.

4.2 Evaluation on unseen data

How the unseen data were obtained. In a new session on 29 October 2026, separate from the training recordings, I recorded **two continuous ≈ 60 s files** that were never used for training or normalisation. Test 1 followed Still → Standing → Walking → Jumping → Still → Standing and Test 2 Standing → Jumping → Walking → Standing → Still → Standing, about 10 s each, so the model is tested on real transitions rather than isolated clips. Ground truth came from the planned schedule, with segment boundaries aligned to the visible changes in the signal; the 1–3 s periods of picking up or putting down the phone were left unlabelled and excluded from the metrics (91 windows decoded, 69 scored).

Table 5 – Performance on the two unseen test recordings (Viterbi decoding)

State (Activity)	Number of Samples (windows)	Sensitivity	Specificity	Accuracy (TP+TN)/N
Still	18	0.944	1.000	0.986
Standing	26	1.000	0.930	0.957
Walking	12	0.917	1.000	0.986
Jumping	13	0.923	1.000	0.986
Overall	**69**			**0.957**

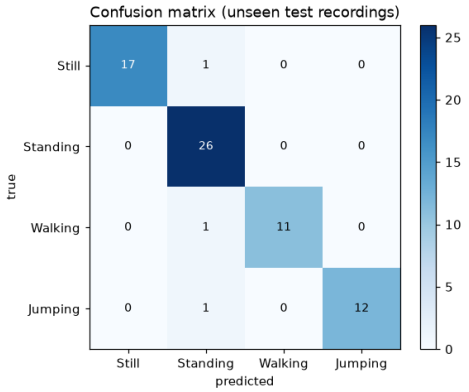


Figure 6 – Confusion matrix on the unseen test recordings (rows: true, columns: predicted).

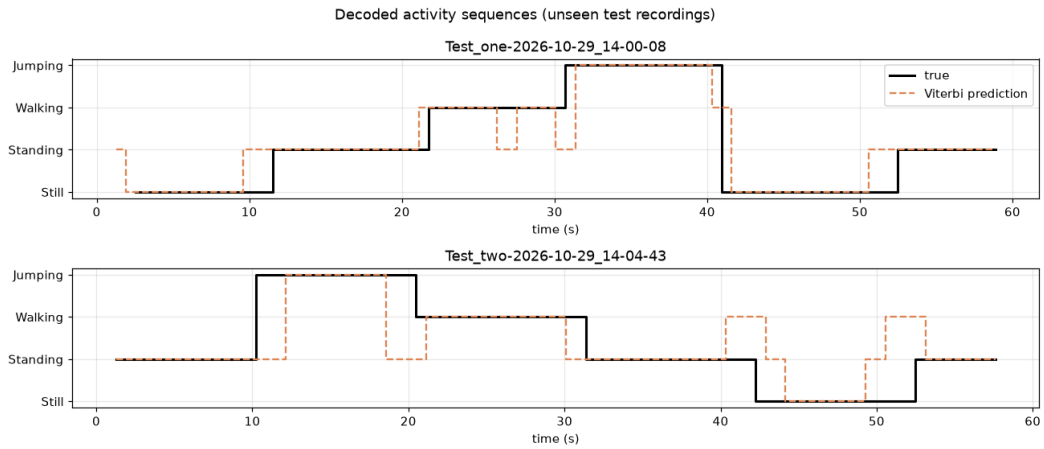


Figure 7 – True (black) vs Viterbi-decoded (orange) activity sequences on the two unseen recordings.

Does the model generalise? Yes. On unseen continuous recordings the HMM reaches **95.7 % accuracy**, with sensitivity ≥ 0.92 and specificity ≥ 0.93 for every activity (validation accuracy on held-out clips: 100 %). Only 3 of 69 windows are

wrong, and all three were predicted as Standing (Figure 6). Figure 7 shows the decoded sequence following every activity change, typically lagging by about one window. The first evaluation was less successful: without drift removal, accuracy was 76.8 % and Still sensitivity only 0.22, because after I put the phone down in Test 1 the linear accelerometer reported a decaying offset of ≈ 1.6 m/s² while the gyroscope showed no motion. None of the training clips contained a put-down, so the model had never seen this artefact and called it Standing. Detrending each window fixed it (76.8 % → 95.7 %). Because this change was motivated by the test files, the 95.7 % figure is slightly optimistic and should ideally be confirmed on a third recording.

5. Discussion and Conclusion

- **Easiest and hardest activities.** Jumping and Still were the easiest: their spectral energy differs by six orders of magnitude and both have specificity 1.00, meaning nothing else was ever mistaken for them. Standing was the hardest. It sits between Still and Walking on every intensity feature, so all three uncertain windows (a Still window overlapping a pick-up, a slowed walking step, the onset of a jumping bout) fell into it, giving the lowest specificity (0.93).
- **Realism of the transition probabilities.** The large diagonal reflects that people keep doing an activity for several seconds. The off-diagonal entries follow real behaviour: Still → Standing (0.11) is picking the phone up; Standing leads to Walking or Jumping (0.12 / 0.09); Walking and Jumping switch between each other (0.14 / 0.14); and Jumping → Still is 0.00, because nobody puts a phone down mid-jump. The learned stay durations (4–8 s) are much shorter than in daily life, because they come from 5–10 s clips chained in a designed order. A deployed tracker would need A re-estimated from long, natural recordings.
- **Effect of sensor noise and sampling rate.** The largest error source was not random noise but a systematic artefact of Android's gravity-removal filter, which shows that features must be robust to the processing inside the phone. The sensor noise floor (≈ 0.01 m/s²) sets the Still feature values. The 50 Hz rate was more than enough; the real limitation is the 2.56 s window, which blurs transitions and causes the one-window lag, and the pick-up motion, which is decoded as Walking because no training state represents it. I also observed that the model before Baum–Welch scored 97.1 % on the test files versus 95.7 % after: EM maximises likelihood, not label accuracy, and with only 69 test windows one window equals 1.4 points.
- **Improvements.** Record more participants and phones, and other phone positions (pocket, hand); collect long natural sessions so that A and π reflect real behaviour; add a 'handling' state for picking up and putting down; add magnetometer or gravity-direction features; use a shorter hop or a hidden semi-Markov model for sharper transitions and realistic durations; and test on more unseen sessions in different environments.

Conclusion. A from-scratch Gaussian HMM with nine time- and frequency-domain features recognises Still, Standing, Walking and Jumping from a single smartphone with 95.7 % accuracy on unseen continuous recordings. The most valuable lesson came from the failure on the first test run: a model is only as reliable as its understanding of the sensor's own processing, and testing on realistic, transition-rich data is what exposed it.